

Internship Project Report

Title: *Image-Based Steganography Tool with Drag-and-Drop GUI*

Submitted by: *Dhyeyaja Petkar*

Submitted to: *Elevate Labs*

Duration: *4th August 2025 – 18th September 2025*

Abstract

This project focuses on building a steganography tool that allows users to hide sensitive text or files inside images using the Least Significant Bit (LSB) technique. The system features a graphical user interface (GUI) with drag-and-drop support, enabling easy selection of files and images. It also includes functionalities like image capacity calculation, verification of embedding, and decoding of hidden messages. This project aims to provide an intuitive and effective way to transmit data securely by concealing its presence.

Introduction

In today's digital world, data security is a major concern. While encryption protects the content, it does not hide the existence of data, which can still draw attention. Steganography, on the other hand, conceals the data inside another medium, such as an image, making it less likely to be detected.

This project implements image-based steganography using the Least Significant Bit (LSB) approach, where the message or file data is hidden within the pixel values of an image without visibly altering it. The solution includes a Python-based GUI, making it user-friendly and practical for non-technical users.

Objectives

- To create a Python tool for hiding text or file data inside an image.
- To implement the LSB steganography algorithm for data embedding and extraction.
- To design a GUI with drag-and-drop support for seamless user experience.
- To provide an image capacity calculator to ensure the message fits in the selected image.
- To verify embedding and decoding through debug logs.

Tools and Technologies

- **Programming Language:** Python 3.13
- **Libraries Used:**
 - Pillow – Image processing
 - tkinter – GUI creation
 - tkinterDnD2 – Drag-and-drop functionality
- **Platform:** Windows 10/11
- **IDE:** Visual Studio Code.

System Design

Architecture

The tool has two main modules:

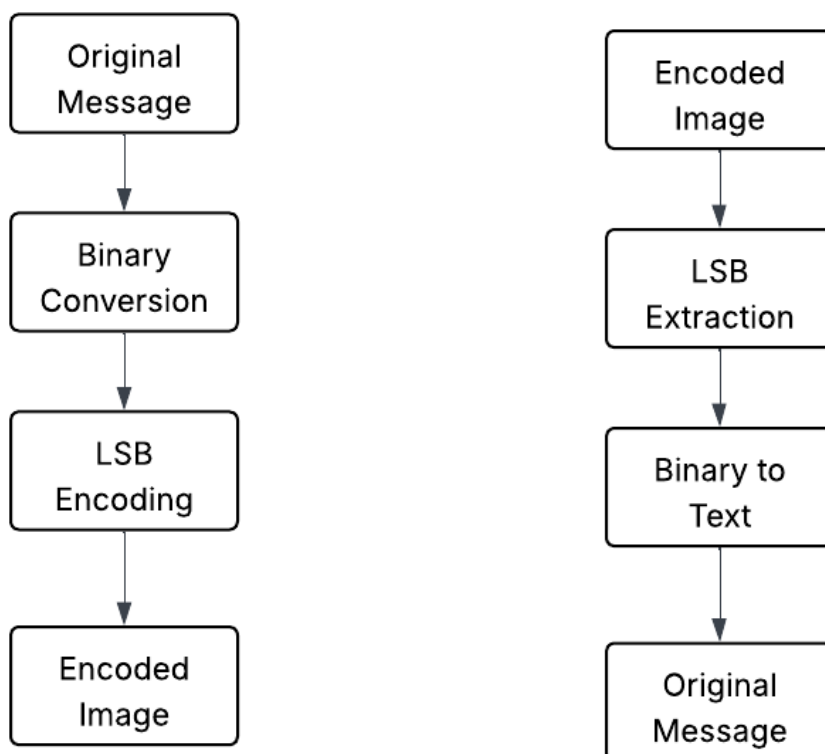
1. Encoding Module

- Inputs: Original Image, Text/File.
- Process:
 - Convert the message/file content to binary.
 - Embed the binary data into the image's pixels using the LSB method.
 - Save the new encoded image.
- Output: Encoded Image

2. Decoding Module

- Inputs: Encoded Image.
- Process:
 - Extract LSB data from the image.
 - Convert binary data back to text or save as a file.
- Output: Original Message (Text File).

Workflow Diagram



Module 1: Encoding Module

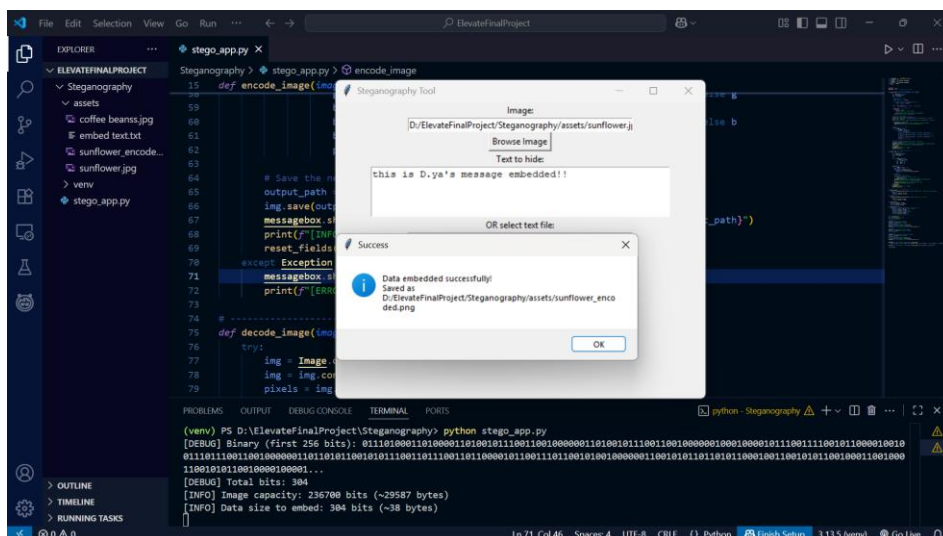
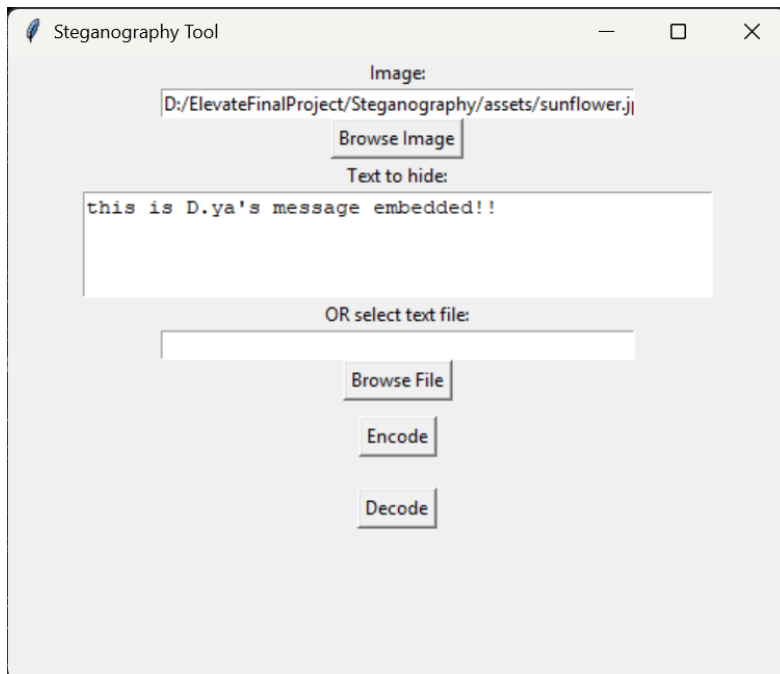
Module 2: Decoding Module

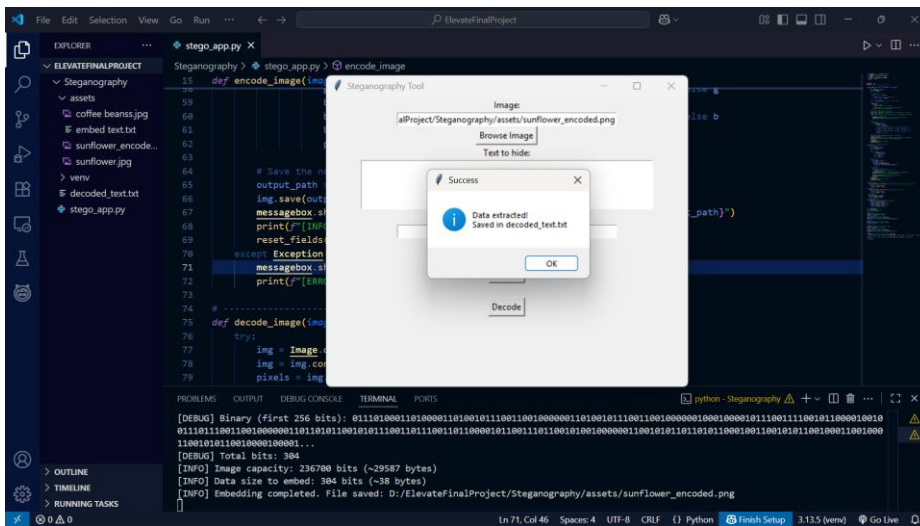
Implementation Details

Key Features Implemented

- **LSB Steganography Algorithm** – Embeds binary data into the least significant bits of image pixels.
- **Support for Text and Files** – Users can hide plain text or an entire file inside an image.
- **Drag-and-Drop GUI** – Built with tkinter and tkinterDnD2, allowing users to drag and drop images or files into the application.
- **Capacity Calculator** – Displays the maximum number of characters or file size that the selected image can hold.
- **Field Reset** – Clears the input fields after each encode/decode operation.
- **Verification Logs** – Prints binary representation and embedding status in the terminal for debugging.

Screenshots





Detailed screenshots of the project are shared in outputs folder on github.

Testing and Results

- **Test Cases Conducted:**
 - Embedded short text messages in small PNG, JPG and BMP image.
 - Embedded text files of different sizes to check capacity limits.
 - Verified decoding accuracy for all test cases.
- **Observations:**
 - Encoded image size increases slightly (due to metadata and file handling).
 - No visible distortion in images after embedding.
- **Performance:**
 - Encoding and decoding processes execute within seconds for standard image sizes.

Future Enhancements

- Add **encryption layer** for additional security.
- Provide **support for more image formats (JPEG, TIFF)**.
- Implement **batch processing** for multiple files.
- Add **progress bar** for large files during encoding.
- Create **CLI support** for advanced users.

Conclusion

The developed tool successfully implements an LSB-based steganography system with a user-friendly GUI and drag-and-drop feature. It allows users to hide and retrieve messages or files within images effectively. The application can be useful in scenarios where confidential data needs to be transmitted or stored without attracting attention.